

```
# =====
# STUDENT PERFORMANCE ANALYTICS & AI-BASED FINAL GRADE PREDICTION
# =====
#
# Student: ABHINAV KUMAR
# Institute: Bansal Institute of Science and Technology, Bhopal
#
# AICTE | IBM SkillsBuild Data Analytics with AI Internship 2026
# Organization: BharatCares
#
# Project Submission Deadline: 24 September 2026
#
# Dataset:
# UCI Student Performance Dataset
# https://archive.ics.uci.edu/dataset/320/student+performance
# =====
```

```
!pip install -q ucimlrepo
```

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRe

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

sns.set_theme(style="whitegrid")

RANDOM_STATE = 42
```

```
print("All libraries imported successfully.")
```

```
# Load the official UCI Student Performance dataset

student_performance = fetch_ucirepo(id=320)

X = student_performance.data.features.copy()
y = student_performance.data.targets.copy()

# Combine features and target
df = pd.concat([X, y], axis=1)

print("Dataset loaded successfully.")
print("Dataset shape:", df.shape)
```

```
print("First five records:")
display(df.head())

print("\nDataset shape:")
print(df.shape)

print("\nNumber of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\nColumn names:")
print(df.columns.tolist())
```

```
print("Data Types:")
display(df.dtypes.to_frame("Data Type"))

print("\nMissing Values:")
missing_values = df.isnull().sum().to_frame("Missing Values")
display(missing_values)

print(
    "\nTotal missing values:",
    df.isnull().sum().sum()
)
```

```
duplicate_count = df.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)

if duplicate_count > 0:
    df = df.drop_duplicates().reset_index(drop=True)
    print("Duplicates removed.")
```

```
print("Dataset shape after duplicate handling:", df.shape)
```

```
print("Descriptive Statistics")

display(
    df.describe(include="all").T
)
```

```
print("Final Grade (G3) Statistics")
print("=" * 40)

print("Mean:", round(df["G3"].mean(), 2))
print("Median:", round(df["G3"].median(), 2))
print("Minimum:", df["G3"].min())
print("Maximum:", df["G3"].max())
print("Standard Deviation:", round(df["G3"].std(), 2))
```

```
plt.figure(figsize=(10, 6))

sns.histplot(
    df["G3"],
    bins=21,
    kde=True,
    color="royalblue"
)

plt.title(
    "Distribution of Final Grade (G3)",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Final Grade (G3)")
plt.ylabel("Number of Students")

plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="studytime",
    y="G3",
    color="skyblue"
)

plt.title(
```

```
        "Study Time vs Final Grade",
        fontsize=16,
        fontweight="bold"
    )

    plt.xlabel("Study Time Category")
    plt.ylabel("Final Grade (G3)")

    plt.tight_layout()
    plt.show()
```

```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="G1",
    y="G3",
    color="darkorange",
    alpha=0.7
)

plt.title(
    "First Period Grade (G1) vs Final Grade (G3)",
    fontsize=15,
    fontweight="bold"
)

plt.xlabel("G1 - First Period Grade")
plt.ylabel("G3 - Final Grade")

plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="G2",
    y="G3",
    color="seagreen",
    alpha=0.7
)

plt.title(
    "Second Period Grade (G2) vs Final Grade (G3)",
    fontsize=15,
    fontweight="bold"
)

plt.xlabel("G2 - Second Period Grade")
plt.ylabel("G3 - Final Grade")
```

```
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="absences",
    y="G3",
    color="crimson",
    alpha=0.7
)

plt.title(
    "Absences vs Final Grade",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Number of Absences")
plt.ylabel("Final Grade (G3)")

plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 6))

sns.boxplot(
    data=df,
    x="sex",
    y="G3",
    palette="Set2"
)

plt.title(
    "Final Grade Distribution by Gender",
    fontsize=15,
    fontweight="bold"
)

plt.xlabel("Gender")
plt.ylabel("Final Grade (G3)")

plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 6))

sns.boxplot(
```

```

data=df,
    x="internet",
    y="G3",
    palette="Set3"
)

plt.title(
    "Final Grade Distribution by Internet Access",
    fontsize=15,
    fontweight="bold"
)

plt.xlabel("Internet Access at Home")
plt.ylabel("Final Grade (G3)")

plt.tight_layout()
plt.show()

```

```

numeric_df = df.select_dtypes(include=np.number)

correlation_matrix = numeric_df.corr()

plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation_matrix,
    cmap="coolwarm",
    center=0,
    annot=True,
    fmt=".2f",
    linewidths=0.5
)

plt.title(
    "Correlation Matrix",
    fontsize=17,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

```

```

g3_correlations = (
    correlation_matrix["G3"]
    .drop("G3")
    .sort_values(
        key=lambda x: x.abs(),
        ascending=False
    )
)

```

```

        print("Correlation of Numerical Vari
        print("=" * 60)

        display(
            g3_correlations.to_frame(
                name="Correlation with (
            )
        )
    )

```

```

print("Top numerical associations with Final Grade")
print("=" * 50)

for feature, correlation in g3_correlations.head(5).items():
    print(
        f"{feature}: {correlation:.3f}"
    )

```

```

print("=" * 65)
print("AI-ASSISTED ANALYTICAL INSIGHTS")
print("=" * 65)

strongest_feature = g3_correlations.index[0]
strongest_correlation = g3_correlations.iloc[0]

print(
    f"\n1. The numerical variable with the strongest "
    f"absolute association with G3 is '{strongest_feature}', "
    f"with a correlation of {strongest_correlation:.3f}."
)

print(
    "\n2. G1 and G2 are previous-period grades and are "
    "expected to contain useful information about G3."
)

print(
    "\n3. Study time, failures and absences can be "
    "as additional student-related variables a "
    "with academic performance."
)

print(
    "\n4. Correlation represents stati "
    "and does not prove causation."
)

print(
    "\n5. Machine-learning pre "
    "on patterns in the tr "
    "treated as guarar

```

)

```
# G3 is the prediction target.
#
# G1 and G2 are deliberately excluded from the primary
# prediction experiment because they are previous-period
# grades and are strongly related to the final grade.

X_model = df.drop(
    columns=["G3", "G1", "G2"]
)

y_model = df["G3"]

print("Feature matrix shape:", X_model.shape)
print("Target shape:", y_model.shape)

print("\nFeatures used for prediction:")
print(X_model.columns.tolist())
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_model,
    y_model,
    test_size=0.20,
    random_state=RANDOM_STATE
)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))

)
```

```
categorical_features = X_model.select_dtypes(
    include=["object", "category"]
).columns.tolist()

numerical_features = X_model.select_dtypes(
    include=np.number
).columns.tolist()

print("Numerical features:")
print(numerical_features)

print("\nCategorical features:")
print(categorical_features)

)
```

```
models = {
    "Linear Regression": LinearRegression(),
```

```

        "Decision Tree": DecisionTreeRegressor(
            max_depth=6,
            random_state=RANDOM_STATE
        ),

        "Random Forest": RandomForestRegressor(
            n_estimators=300,
            max_depth=10,
            random_state=RANDOM_STATE
        ),

        "Gradient Boosting": GradientBoostingRegressor(
            n_estimators=300,
            max_depth=6,
            random_state=RANDOM_STATE
        )
    }

    # Create a dictionary to store the trained models
    trained_models = {}

    # Loop through the models and train them
    for model_name, estimator in models.items():
        # Create a pipeline with the estimator and the preprocessor
        pipeline = Pipeline(
            steps=[
                (
                    "preprocessor",
                    preprocessor
                ),
                (
                    model_name,
                    estimator
                )
            ]
        )

        # Train the pipeline
        pipeline.fit(X_train, y_train)

        # Store the trained pipeline in the dictionary
        trained_models[model_name] = pipeline

    # Return the dictionary of trained models
    return trained_models

```

```

results = []
trained_models = {}

for model_name, estimator in models.items():

    pipeline = Pipeline(
        steps=[
            (
                "preprocessor",
                preprocessor
            ),
            (
                model_name,
                estimator
            )
        ]
    )

    # Train the pipeline
    pipeline.fit(X_train, y_train)

    # Store the trained pipeline in the dictionary
    trained_models[model_name] = pipeline

# Return the dictionary of trained models
return trained_models

```

```
plt.figure(figsize=(11, 6))

sns.barplot(
    data=results_df,
    x="Model",
    y="R2",
    color="steelblue"
)

plt.title(
    "Machine Learning Model Comparison - R2 Score",
    fontsize=16,
```

```
fontweight="bold"
)

plt.xlabel("Model")
plt.ylabel("R2 Score")

plt.xticks(rotation=20)

plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(11, 6))

sns.barplot(
    data=results_df,
    x="Model",
    y="MAE",
    color="darkorange"
)

plt.title(
    "Machine Learning Model Comparison - MAE",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Model")
plt.ylabel("Mean Absolute Error")

plt.xticks(rotation=20)

plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(11, 6))

sns.barplot(
    data=results_df,
    x="Model",
    y="RMSE",
    color="seagreen"
)

plt.title(
    "Machine Learning Model Comparison - RMSE",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Model")
plt.ylabel("Root Mean Squared Error")
```

```
plt.xticks(rotation=20)
```

```
plt.tight_layout()  
plt.show()
```

```
best_model_name = results_df.iloc[0]["Model"]  
best_model = trained_models[best_model_name]  
  
best_mae = results_df.iloc[0]["MAE"]  
best_rmse = results_df.iloc[0]["RMSE"]  
best_r2 = results_df.iloc[0]["R2"]  
  
print("=" * 60)  
print("MODEL EVALUATION SUMMARY")  
print("=" * 60)  
  
print("Selected model based on highest test-set R2:")  
print(best_model_name)  
  
print("\nMAE:", round(best_mae, 4))  
print("RMSE:", round(best_rmse, 4))  
print("R2:", round(best_r2, 4))
```

```
best_predictions = best_model.predict(X_test)  
  
prediction_comparison = pd.DataFrame({  
    "Actual G3": y_test.values,  
    "Predicted G3": best_predictions  
})  
  
display(  
    prediction_comparison.head(15).round(2)  
)
```

```
plt.figure(figsize=(9, 7))  
  
sns.scatterplot(  
    x=y_test,  
    y=best_predictions,  
    color="purple",  
    alpha=0.75  
)  
  
min_value = min(  
    y_test.min(),  
    best_predictions.min()  
)  
  
max_value = max(  
    y_test.max(),  
    best_predictions.max()  
)
```

```

        y_test.max(),
        best_predictions.max()
    )

    plt.plot(
        [min_value, max_value],
        [min_value, max_value],
        color="red",
        linestyle="--",
        label="Perfect Prediction"
    )

    plt.title(
        f"Actual vs Predicted Final Grade",
        fontsize=14,
        fontweight="bold"
    )

    plt.xlabel("Actual Final Grade")
    plt.ylabel("Predicted Final Grade")

    plt.legend()

    plt.tight_layout()
    plt.show()

```

```

# Predict one student from the test set

sample_student = X_test.iloc[[0]]

actual_score = y_test.iloc[0]

predicted_score = best_model.predict(
    sample_student
)[0]

print("=" * 50)
print("SAMPLE STUDENT PREDICTION")
print("=" * 50)

print("Actual Final Grade:", round(actual_score, 2))
print("Predicted Final Grade:", round(predicted_score, 2))

print(
    "Absolute Prediction Error:",
    round(
        abs(actual_score - predicted_score),
        2
    )
)

```

```

# Feature importance is available for tree-based models.
# We calculate it for the Random Forest model if available.

random_forest_model = trained_models["Random Forest"]

preprocessor_rf = (
    random_forest_model
    .named_steps["preprocessor"]
)

rf_estimator = (
    random_forest_model
    .named_steps["model"]
)

feature_names = (
    preprocessor_rf
    .get_feature_names_out()
)

importance_values = (
    rf_estimator.feature_importances_
)

feature_importance = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance_values
})

feature_importance = (
    feature_importance
    .sort_values(
        by="Importance",
        ascending=
    )
)

di

```

```

plt.figure(figsize=(11, 7))

sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature",
    color="teal"
)

plt.title(
    "Top 15 Random Forest Feature Im
    figsize=16

```

```

        fontsize=10,
        fontweight="bold"
    )

    plt.xlabel("Feature Importance")
    plt.ylabel("Feature")

    plt.tight_layout()
    plt.show()

```

```

feature = feature_importance.iloc[0]["Feature"]
importance = feature_importance.iloc[0]["Importance"]

t("=" * 70)
t("FINAL AI-ASSISTED PROJECT INTERPRETATION")
t("=" * 70)

t(
    f"\nThe evaluated model was {best_model_name}."
)

print(
    f"\nIts test-set R2 score was {best_r2:.3f}, "
    f"with an MAE of {best_mae:.3f} "
    f"and an RMSE of {best_rmse:.3f}."
)

print(
    f"\nWithin the Random Forest model, "
    f"the highest feature importance "
    f"encoded features was associated with "
    f"'{top_feature}'."
)

print(
    "\nThese results describe the model's performance on the
    "selected dataset, which was used as evidence for the model's
    "accuracy.
)

print(
    "\nThe model's performance is judged to be
    "satisfactory.
)

```

COMPLETED")

ABHINAV KUMAR")

ansal Institute of Science and Technology, Bhopal"

```
n: AICTE | IBM SkillsBuild Data Analytics with AI "
nship 2026"
```

```
("Organization: BharatCares")
```

```
("Dataset: UCI Student Performance Dataset")
```

```
("Dataset records:", len(df))
```

```
("Dataset columns:", len(df.columns))
```

```
(
    "\nBest evaluated model:",
    best_model_name
)
```

```
print(
    "Test MAE:",
    round(best_mae, 4)
)
```

```
print(
    "Test RMSE:",
    round(best_rmse, 4)
)
```

```
print(
    "Test R²:",
    round(best_r2, 4)
)
```

```
print("\nAnalysis and mach
```

```
numeric_pipeline = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="median"),
        ),
        (
            "scaler",
            StandardScaler()
        )
    ]
)
```

